



A fully offline, privacy-first semantic file finder — find
any personal document by what it is, not what it's
named.

Project Summary · July 2026

1. The idea

Everyone has the same recurring, small frustration: a file exists somewhere on their computer, but they can't remember its exact name or which folder it's buried in. Windows Search, Spotlight, and manual folder-browsing all rely on matching the *filename* — not what the document is actually *about*.

Findr solves this with semantic search: the user types what they remember ("where is my offer letter for Meritech company", "salary slip", "aadhar card") and the app finds the matching file by meaning, not by exact filename, then opens it directly.

Core promise: fast, accurate, and fully private. No cloud, no accounts, no LLM calls at runtime — everything runs and stays on the user's own machine.

Who it's for

Anyone with a large, messy personal document collection on their own laptop — the same architecture serves very different people without any redesign:

- Individuals with years of scattered personal documents (IDs, certificates, tax records)
- Students with semesters of assignments, lecture notes, and scanned textbook chapters
- HR professionals with hundreds of resumes and offer letters saved locally
- Freelancers/consultants with client contracts and invoices on one machine

All of these are single-user, single-machine use cases — exactly what this architecture is built for. A shared/multi-user "team search" product is a plausible *future* direction, but a meaningfully different (and much larger) build — intentionally out of scope for v1.

2. How it works

End-to-end flow

1. User installs the app and grants permission to specific folders (Documents, Downloads, Desktop, or custom)
2. App scans those folders, extracts text from every supported file (native extraction, with OCR fallback for scanned/image documents)
3. Each document is split into chunks and embedded locally into vectors
4. Vectors + metadata (file path, content hash, filename) are stored in a local vector database
5. User types a query in a single search bar — no chat, no explanation, just results
6. Hybrid semantic + keyword search ranks candidates; results above a similarity threshold are shown (1 clear match, or up to 3 ranked options)
7. User clicks a result — the OS opens the file directly

Handling moved or renamed files

A background file-system watcher tracks moves/renames/deletes live while the app is open, updating the file's stored path instantly with no re-processing needed. For changes that happen while the app is closed, a startup reconciliation step re-hashes files in the watched folders and relinks any entry whose path is no longer valid — using the file's *content*, not its name or location, as the source of truth.

3. Architecture & tech decisions

Layer	Choice	Why
Desktop shell	Tauri (Rust + native webview)	Small binary size, native cross-platform packaging, no Electron bloat
Core logic	Python, as one persistent background process	Fast to build/test/iterate; avoids per-query cold-start by staying warm for the whole app session
Rust ↔ Python bridge	Spawned once, communicates via stdin/stdout JSON	No HTTP server, no port, no SQL/ORM layer needed for a single-user local app
Vector database	Chroma (embedded, local files only)	Zero-server, ideal for a single-user desktop app; HNSW indexing keeps search sub-50ms
Embeddings	Small local ONNX model (e.g. all-MiniLM / bge-small), bundled with the app	No Ollama dependency, no external server, one-click install for the end user
OCR	Tesseract	Free, offline, cross-platform, good enough accuracy for scanned personal documents
File tracking	watchdog (live) + content-hash reconciliation (startup)	Self-healing against moved/renamed files without manual re-indexing

Explicitly not used: FastAPI as a public server, SQLAlchemy/foreign-key relational modeling, cloud sync, or any Claude/LLM API call at runtime. None of these match a single-user, fully offline tool — they were considered and deliberately cut to keep the product lean, fast, and trustworthy.

What actually touches the internet

In the finalized, purely local design: **nothing, ever**, after install. (An earlier version of the idea explored an optional cloud-synced "knowledge box" for a companion webapp — that layer was intentionally set aside to keep v1 fully offline and infrastructure-free.)

4. Performance & footprint

Search speed

With the embedding model and vector DB already warm (persistent background process, no cold start), a typical query — e.g. *"where is my offer letter for Meritech company"* — resolves in roughly **35–75 milliseconds** end-to-end: faster than a human eye-blink, and faster than manually clicking into a single folder.

Initial indexing time (one-time cost)

Library size	Estimated first-scan time
100 files	~10–45 seconds
1,000 files	~1–7 minutes (faster with parallelized OCR)

Disk & memory footprint

Disk

- Installed app: ~220–350 MB
- Vector DB at 1,000 docs: ~20–50 MB
- Vector DB at 5,000 docs: ~100–250 MB

RAM (steady state)

- Tauri shell: ~50–100 MB
- Python daemon + embedding model: ~150–250 MB
- Total: roughly ~250–400 MB

Comparable to running a single chat or music-streaming desktop app — a reasonable footprint on any modern laptop.

5. Build checklist

Phase 1 — Foundation

Project setup · choose Chroma as the vector store · local IPC (no public backend) · choose Tesseract for OCR

Phase 2 — Ingestion pipeline

Folder discovery with permission prompt · whitelist file-type filtering · extract → chunk (500–1000 words) → embed → store with path/hash metadata

Phase 3 — Search

Hybrid semantic + keyword scoring · dedup by file path · similarity threshold gating · ranked results with similarity %

Phase 4 — File integrity tracking

Live watcher for moved/deleted/created files · startup reconciliation via content hash

Phase 5 — App shell

Tauri scaffold · single-search-bar UI · first-run consent screen · indexing progress bar · cross-platform packaging (Windows/macOS/Linux), code signing & notarization

Estimated timeline

Roughly **3–4 weeks** solo, with AI-assisted coding, for a fully working v1 — core engine (ingestion + hybrid search) first, validated against a real messy folder before any UI work, then the Tauri shell and packaging.

6. Positioning & why it's worth building

Where it fits in the market

- **vs. Spotlight / Windows Search** — those are filename/keyword only; Findr understands meaning
- **vs. Windows Recall / Rewind** — those capture broad activity history and raise real privacy concerns; Findr is narrow-scope, local-only, and only touches files the user explicitly grants access to

Honest read

The engineering is genuinely achievable, and AI coding tools make the scaffolding fast — but that also means the code alone isn't the moat. The real value sits in the parts that take real testing and judgment: similarity threshold tuning, reliable handling of moved/renamed files, graceful OCR failure handling, and a trustworthy, transparent install experience. That combination — narrow, fast, and genuinely private — is a believable, differentiated product, not just a demo.

Distribution plan

Direct .dmg / .msi / .AppImage download

No app store (avoids sandbox restrictions)

Show HN / Product Hunt launch

Open-source core for trust

One-time purchase or free tier + paid unlock